Threat Hunting with KQL — Microsoft Defender XDR & Sentinel

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 5
- The problem . . . 5
- The solution . . . 5
- How it works . . . 5
- The boundary — proactive hunting, not reactive response . . . 6
- Important boundaries . . . 6
- Effort . . . 6

Solution Overview — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 6
- What this is . . . 6
- How the technology works (the mental model) . . . 7
- The standard deployment model . . . 8
- Scope — what this solution covers . . . 8
- Out of scope (handled elsewhere) . . . 9
- Authentication and connections used . . . 9

Requirements — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 9
- R1. Microsoft Defender XDR onboarded, with advanced hunting available . . . 10
- R2. A connected Sentinel workspace with the hunting tables populated . . . 10
- R3. Least-privilege, read-only hunting access (the hunter RBAC group) . . . 10
- R4. An agreed hunt backlog and priority (which techniques matter) . . . 11
- R5. A findings-documentation home (hunt register + bookmarks) . . . 11
- R6. A defined hunt-to-incident hand-off (the reactive boundary) . . . 12
- R7. The PowerShell toolchain . . . 12
- R8. Awareness of advanced-hunting limits (schema, retention, quotas) . . . 12

Global Requirements — Sentinel Solutions 13
- G1. An Azure subscription . . . 13
- G2. A Log Analytics workspace . . . 13
- G3. Microsoft Sentinel enabled on the workspace . . . 14
- G4. Administrative roles (least privilege) . . . 14
- G5. Ingestion and retention cost plan . . . 14
- G6. Diagnostic settings capability on each source . . . 14
- G7. Change control & rollback posture . . . 15

Implementation — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 15
- Object naming (used verbatim throughout) . . . 15
- Phase 0 — Assess (is the hunting surface usable?) . . . 16
- Phase 1 — Configure (RBAC, library, register, cadence) . . . 17

Phase 2 — Pilot (run hypothesis-driven hunts, read-only, and document) . . . 18
Phase 3 — Hand off confirmed findings to incident response (the reactive boundary) . . . 18
Phase 4 — Validate (coherent results + a clean promotion) . . . 19
Phase 5 — Expand (grow the library, run the cadence) . . . 19
Rollback (for any step) . . . 20
Testing & Validation — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 20
Gate 1 — The hunting surface has data (both platforms) . . . 20
Gate 2 — Read-only hunter access is in place (and only read-only) . . . 20
Gate 3 — The KQL hunt library is saved and MITRE-tagged on both surfaces . . . 21
Gate 4 — The library runs clean on both platforms . . . 21
Gate 5 — Pilot hunts are documented (hypothesis → outcome → decision) . . . 21
Gate 6 — Findings are preserved as bookmarks with entities . . . 22
Gate 7 — Confirmed true positives are handed off (not acted on here) . . . 22
Gate 8 — At least one successful hunt is promoted to a detection . . . 22
Gate 9 — The cadence and trend are established . . . 22
Troubleshooting — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 23
A hunt returns nothing (and you expected activity) . . . 23
A hunt returns far too much to triage . . . 23
A hunt errors on a missing column or unknown table . . . 23
Results look truncated / a limit was hit . . . 24
`Invoke-MgSecurityRunHuntingQuery` throttles or errors . . . 24
The hunter group can change things (or cannot query) . . . 24
A bookmark loses its evidence or entities . . . 24
A confirmed true positive — but no clean hand-off . . . 25
A promoted detection floods the queue . . . 25
PowerShell / connection errors . . . 25
Best Practices — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 25
Hunt from a hypothesis, not a hunch . . . 26
Hunt against a map (MITRE ATT&CK;) . . . 26
Stay read-only — hunting finds, it does not act . . . 26
Document so the knowledge compounds . . . 26
Promote wins into permanent detections . . . 27
Run it as a program, on a cadence . . . 27
Where to go next . . . 27
Deliverables Summary — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel 28
Documentation . . . 28

Hunt library & PowerShell tools (5) . . . 29
Named objects (verbatim — used across docs, scripts, and tests) . . . 30
What this solution deliberately does NOT do . . . 30
How it is delivered . . . 31
Appendix A — PowerShell Scripts 32

Executive Summary — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

The problem

A security team that only *waits for alerts* is always a step behind. Detections fire on the attacks someone already anticipated and wrote a rule for — but the intrusions that matter most are the ones that slip **between** the rules: a living-off-the-land command that looks like admin work, a mailbox rule quietly forwarding to an attacker, a service account signing in from a place it never has. By the time a reactive alert catches up, the adversary has usually already moved.

Two failure modes are common, and most organizations have both:

- **No proactive discipline.** Nobody systematically asks "if an attacker were already inside, how

would we find them?" There is no repeatable method, no hypothesis backlog, no cadence — hunting happens only after an incident, if at all.

- **Knowledge that evaporates.** When someone *does* run a clever query and finds something, the query

lives in a browser tab and the finding lives in a chat message. The next hunter starts from zero, and a hunt that worked never becomes a permanent detection.

The solution

This solution stands up a **proactive, repeatable threat-hunting program** across **Microsoft Defender XDR advanced hunting** and **Microsoft Sentinel**, driven by **KQL**. It:

- **Establishes the hunting environment** — advanced hunting (XDR) and Sentinel hunting, with scoped,

least-privilege **read-only** access so hunters can query broadly without the power to change anything.

- **Delivers a MITRE ATT&CK-aligned, reusable KQL hunt library** — real, ready-to-run hunt queries,

each tagged to the tactic/technique it targets, so the team hunts against a **map of adversary behavior**, not a blank page.

- **Runs a hypothesis-driven method on a cadence** — every hunt starts from a written hypothesis, is

scoped and time-boxed, and produces a documented result (found / not found / needs follow-up), captured as **bookmarks** and a **hunt register**.

- **Promotes successful hunts into permanent detections** — a hunt that reliably finds real activity is

turned into a **scheduled analytics rule** (Sentinel) or a **custom detection rule** (Defender XDR), so the win is captured forever instead of re-run by hand.

- **Hands confirmed findings off cleanly** — a true-positive hunt becomes an **incident** that flows

into your **reactive incident-response process**, not an ad-hoc fire drill.

How it works

It follows the same repeatable model every solution in this program uses — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. You first **assess** the hunting surface (which tables have data, who has access, which techniques matter most), **configure** the hunt library, register, RBAC, and cadence, run a **pilot** batch of hypothesis-driven hunts read-only and document what you find, **validate** that findings are coherent and that a successful hunt promotes cleanly into a detection, and only then **expand** the library across the ATT&CK matrix on a recurring cadence. **Every hunt is read-only** — hunting *finds*; it never *acts*.

The solution ships as **self-guided, click-by-click documentation plus a KQL hunt library and four PowerShell tools** (Microsoft.Graph `runHuntingQuery` for Defender XDR + Az for Sentinel) that discover the hunting surface, run the whole library across both platforms and report findings, validate the program against pass/fail gates, and report hunt-program metrics — all **read-only**, with **no credentials in code** and **no response actions**.

The boundary — proactive hunting, not reactive response

This solution is deliberately the **proactive** half of detection & response. It **finds** suspicious activity and turns durable findings into detections; it does **not** contain, isolate, disable, or remediate. **Response actions — device isolation, account disablement, mailbox purges, escalation, and crisis coordination — are out of scope and belong to the companion *M365 & Azure Incident Response & Escalation Runbook*.** When a hunt confirms a true positive, this solution's job is to **document it and hand it off** to that reactive process, cleanly. The two are designed to fit together: hunting feeds the incident queue; the runbook works the queue.

Important boundaries

Threat hunting **reduces dwell time; it is not a guarantee** — a hunt that returns nothing means *this hypothesis found nothing over this window*, not that the environment is clean. It **assumes telemetry is already flowing**: a hunt can only find what the platforms ingest, so it depends on onboarded Defender XDR data and a connected Sentinel workspace. It **does not decide** which techniques matter most for your organization or your risk tolerance — those are security-leadership determinations that prioritize the hunt backlog. Defender XDR advanced hunting **schema, retention, and query limits**, and the availability of specific tables and custom-detection features, **evolve and can differ by cloud (Commercial vs Government) and by license** — *verify current availability and limits in your tenant.*

Effort

A competent security analyst can stand up the hunting environment, import the library, and run and document the first pilot hunts in a focused engagement, then settle into a recurring monthly cadence. The documentation is click-by-click; the KQL library and PowerShell tools do the heavy lifting. Delivered as a **Complete Solution Package (PDF + Word + scripts)** — reusable on every tenant and every hunt cycle.

Solution Overview — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

What this is

A complete, self-guided method for standing up a **proactive threat-hunting program** across **Microsoft Defender XDR advanced hunting** and **Microsoft Sentinel**, driven by **KQL**. It gives you a **MITRE ATT&CK-aligned, reusable hunt library**, a **hypothesis-driven method and cadence**, a way to **document findings** (bookmarks + a hunt register), and a path to **promote successful hunts into permanent detections**.

It is the **proactive** layer of a security operations program. It assumes telemetry is already flowing (Defender XDR onboarded, a connected Sentinel workspace) and builds hunting on top of it. It is the counterpart to two adjacent solutions: the *Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage* solution (which tunes the **reactive** scheduled detections), and the *M365 & Azure Incident Response & Escalation Runbook* (which **responds** once something is confirmed). This solution sits before both: it **finds** what the rules miss and turns durable findings into new detections and new incidents.

How the technology works (the mental model)

- **Advanced hunting (Microsoft Defender XDR).** A query-based investigation tool in the Defender

portal (`security.microsoft.com`) that runs **KQL** over a **unified schema** of Defender tables — device events (`DeviceProcessEvents`, `DeviceNetworkEvents`, `DeviceFileEvents`, …), identity (`IdentityLogonEvents`, `IdentityInfo`), email/collaboration (`EmailEvents`, `CloudAppEvents`), and alerts (`AlertEvidence`, `AlertInfo`). Data is typically retained for **~30 days** and queries have **row/time limits** — *verify the current schema, retention, and limits in your tenant.*

- **Microsoft Sentinel hunting.** Sentinel adds a **Hunting** experience over the Log Analytics

workspace: saved **hunt queries** (MITRE-tagged), ad-hoc KQL in **Logs**, **bookmarks** to preserve interesting rows, **Livestream** to watch a query in near-real-time, and (where available) a **Hunts** feature to organize a hunt as an investigation with hypotheses and linked bookmarks. Sentinel queries the workspace tables (`SigninLogs`, `AuditLogs`, `OfficeActivity`, `Device*` if the Defender connector streams them, `CommonSecurityLog`, …), often over **longer retention** than advanced hunting.

- **KQL as the shared language.** Both surfaces speak **KQL**, so a hunt is fundamentally a KQL

hypothesis — "if technique *X* were happening, this query would surface it." The **same idea** often runs on both platforms with different table names; the delivered library notes which surface each hunt targets.

- **MITRE ATT&CK alignment.** ATT&CK is a matrix of adversary **tactics** (the "why" — e.g. *Execution*,

Collection, *Command and Control*) and **techniques** (the "how" — e.g. *T1059 Command and Scripting Interpreter*). Tagging each hunt to a technique turns the library into a **coverage map**: you can see which tactics you actively hunt and where you have never looked.

- **Hypothesis-driven hunting.** A good hunt is not "run a query and see." It is: **state a hypothesis**

→ **scope it** (time, assets, identities) → **run the KQL** → **triage the results** (benign / suspicious / confirmed) → **document the outcome** → **decide the follow-up** (promote to a detection, hand off as an incident, or record "not found this window").

- **Bookmarks & the hunt register.** A **bookmark** preserves a specific result row (with its entities)

so it is not lost when the query window rolls off — the raw evidence of a finding. The **hunt register** is the human record: hypothesis, scope, query, outcome, and decision, per hunt.

- **Promotion: hunt → detection.** A hunt that reliably finds real, actionable activity is **promoted**

into a permanent, automatic detection — a **scheduled analytics rule** in Sentinel, or a **custom detection rule** in Defender XDR (built directly from an advanced-hunting query). The one-off hunt becomes standing coverage.

- **Hunt → incident hand-off (read-only boundary).** When a hunt confirms a true positive, it becomes an

incident that flows into the **reactive** process. This solution **stops at the hand-off** — it never isolates, disables, or remediates. Those response actions live in the companion *M365 & Azure Incident Response & Escalation Runbook*.

The standard deployment model

Every solution in this program follows the same five stages:

1. **Assess.** Confirm the hunting surface is usable — which advanced-hunting tables and Sentinel tables

have recent data, who holds hunting access, and which ATT&CK techniques are the priority. Inventory any existing hunt queries, bookmarks, and custom detections (run `THREATHUNT-Discovery.ps1`).

1. **Configure.** Stand up the program's named objects: the **read-only hunter RBAC group**, the

reusable KQL hunt library (`Hunt Library - Tenant - Main`), the **hunt register**, and the **monthly cadence** — naming every object.

1. **Pilot.** Run the first batch of hypothesis-driven hunts **read-only** across both platforms,

triage the results, and **document findings** as bookmarks + register entries. Hand any confirmed true positive off to incident response (run `THREATHUNT-RunHuntLibrary.ps1`, `THREATHUNT-Report.ps1`).

1. **Validate.** Confirm the hunts return coherent results, findings are documented, and at least one

successful hunt **promotes cleanly** into a scheduled analytics rule / custom detection (run `THREATHUNT-Validate.ps1`).

1. **Expand.** Promote the hunts worth keeping, grow the library across the rest of the ATT&CK matrix,

and run the loop on the agreed cadence.

Scope — what this solution covers

- **The hunting environment** — Defender XDR advanced hunting + Sentinel hunting, scoping, and

least-privilege, read-only RBAC for hunters.

- **A MITRE ATT&CK-aligned reusable KQL hunt library** — real hunt queries, tagged to technique,

runnable on both surfaces.

- **A hypothesis-driven hunt method and cadence** — hypothesis → scope → run → triage → document →

decide, on a recurring schedule.

- **Findings documentation** — bookmarks, a hunt register, and the **hunt-to-incident hand-off**.
- **Promotion of successful hunts** into scheduled analytics rules / custom detection rules.

Out of scope (handled elsewhere)

- **Incident response & escalation** — containment, isolation, account disablement, mailbox purge,

eradication, and crisis coordination are the **companion *M365 & Azure Incident Response & Escalation Runbook***. This solution hands confirmed findings **to** that process and stops.

- **Standing up the workspace, connectors, and ingestion** — a hunt can only find what is ingested;

connect sources first (the *Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion* solution).

- **Full detection tuning & false-positive engineering** for the promoted rules — the *Microsoft

Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage* solution owns rule tuning; this solution performs the **promotion** and points there for the tuning loop.

- **Automated response / SOAR playbooks** — a separate Sentinel automation solution.
- **The determination of which techniques matter most and your risk tolerance** — security-leadership

decisions that set the hunt backlog priority; this solution provides the method and the library to execute against them.

Authentication and connections used

- **Microsoft Defender portal** (`security.microsoft.com`) → **Advanced hunting** (KQL) and **Custom

detection rules** (promotion).

- **Microsoft Sentinel** (Azure portal `portal.azure.com` or the unified Defender portal) → **Hunting**,

Logs, **Bookmarks**, **Livestream**, and **Analytics** (promotion).

- **PowerShell** with **Microsoft.Graph** (`Microsoft.Graph.Security` — `Invoke-MgSecurityRunHuntingQuery`)

to run advanced-hunting KQL against Defender XDR, and the **Az** modules (`Az.Accounts`, `Az.OperationalInsights` for `Invoke-AzOperationalInsightsQuery`, `Az.SecurityInsights` for hunts, bookmarks, and rules) for Sentinel.

- **KQL** — the delivered `Hunt Library - Tenant - Main` (`THREATHUNT-HuntLibrary.kql`), run on both

surfaces.

- All tools authenticate **interactively** with an MFA-capable admin, need only **read/hunting** scopes

(`ThreatHunting.Read.All` for Graph; Security Reader / Microsoft Sentinel Reader for the portals), and hold **no credentials in code**. Government-cloud tenants pass the matching environment / Graph endpoint — *verify the correct endpoints and scopes for your cloud.*

Requirements — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

These are the **solution-specific** prerequisites for running a proactive hunt program. The **platform baseline** — an Azure subscription, a Log Analytics workspace, Sentinel enabled, least-privilege roles, an ingestion/retention plan, diagnostic-settings capability, and change control — is covered once in `global-requirements-sentinel-solutions.md` (**G1–G7**). Read that first; the items below layer on top of it and are not repeated here.

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing**.

R1. Microsoft Defender XDR onboarded, with advanced hunting available

• **Description:** Microsoft Defender XDR (Defender for Endpoint / Identity / Office 365 / Cloud Apps as applicable) is onboarded so its telemetry populates the **advanced hunting** schema (e.g. `DeviceProcessEvents`, `DeviceNetworkEvents`, `IdentityLogonEvents`, `EmailEvents`, `CloudAppEvents`).

• **Why required:** Roughly half the delivered hunt library runs on the Defender XDR side; those hunts can only find what Defender ingests. Advanced hunting also requires the appropriate license (commonly Microsoft 365 E5 / Defender for Endpoint Plan 2 or equivalent) — *verify current licensing in your tenant.*

• **How to verify:** `security.microsoft.com` → **Hunting** → **Advanced hunting** → run `DeviceProcessEvents | where Timestamp > ago(1d) | count` → rows return. `THREATHUNT-Discovery.ps1` also reports which advanced-hunting tables have recent data.

• **How to implement if missing:** Onboard the relevant Defender XDR workloads and endpoints; confirm licensing; wait for telemetry to populate before hunting on that table.

R2. A connected Sentinel workspace with the hunting tables populated

• **Description:** The Sentinel-side tables the library queries — e.g. `SigninLogs`, `AuditLogs`, `OfficeActivity`, `CommonSecurityLog`, and the Defender `Device*` tables **if** the Defender connector streams them to the workspace — have recent data.

• **Why required:** The Sentinel-side hunts (and longer-retention hunts beyond advanced hunting's ~30 days) run against the workspace. A hunt against an empty table silently finds nothing.

• **How to verify:** Sentinel → **Logs** → `SigninLogs | where TimeGenerated > ago(1d) | count` (repeat per table). `THREATHUNT-Discovery.ps1` reports freshness for the expected Sentinel tables.

• **How to implement if missing:** Connect the missing sources first — use the *Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion* solution. Note which platform holds which table so you run each hunt on the surface that has the data.

R3. Least-privilege, read-only hunting access (the hunter RBAC group)

• **Description:** A scoped access model that lets hunters **query broadly but change nothing** — a security group `Threat Hunters - Tenant - Group` granted **Security Reader** (Defender portal / advanced hunting) and **Microsoft Sentinel Reader** (workspace). Promotion of a hunt to a detection is a separate, elevated action performed by a smaller set of admins (Sentinel Contributor / Defender security settings), not by every hunter.

• **Why required:** Hunting is inherently read-only; granting hunters standing write/response power violates least privilege and widens blast radius. Separating **hunt** (read) from **promote** (write) keeps the program safe and auditable.

• **How to verify:** In Entra → **Groups**, confirm `Threat Hunters - Tenant - Group` exists with the intended members; on the workspace **Access control (IAM)** it holds **Microsoft Sentinel Reader**; in the Defender portal the members have **Security Reader**.

• **How to implement if missing:** Create the group; assign **Microsoft Sentinel Reader** at workspace/resource-group scope and **Security Reader** in the Defender portal; keep a separate, smaller admin set for promotion. Prefer PIM-eligible activation for the elevated promotion role where Entra ID P2 is available — *verify licensing.*

R4. An agreed hunt backlog and priority (which techniques matter)

• **Description:** A short, written statement from security leadership of **which ATT&CK tactics/techniques matter most** for your organization (crown-jewel systems, likely adversaries, recent threat intel), used to order the hunt backlog.

• **Why required:** You cannot hunt everything at once; hypothesis-driven hunting needs a **prioritized backlog**. **Which techniques to hunt first is a** risk decision**, not a technical default.

• **How to verify:** Confirm a written, owned priority list exists (even a ranked shortlist of ATT&CK techniques is enough to start).

• **How to implement if missing:** Facilitate a short session to rank techniques by risk (map crown-jewel assets and current threat intel to ATT&CK). This is a **leadership determination**; the solution supplies the library and method to execute against it.

R5. A findings-documentation home (hunt register + bookmarks)

• **Description:** A place to record each hunt's **hypothesis, scope, query, outcome, and decision** — the `Hunt Register - Tenant - Main` (a document, wiki page, or tracking list) — plus the use of **Sentinel bookmarks** to preserve evidence rows.

• **Why required:** A hunt whose result lives in a browser tab is lost. Documentation is what turns hunting from a one-off into a **program** — it prevents re-hunting the same ground and is the raw material for promotion and for the incident hand-off.

• **How to verify:** Confirm the register exists and has an owner; confirm hunters know to bookmark findings in Sentinel (**Hunting/Logs** → **Add bookmark**).

• **How to implement if missing:** Create the register (start from the template referenced in `03-implementation.md`); assign an owner; establish the bookmark convention `Bookmark-<Hunt>-Finding`.

R6. A defined hunt-to-incident hand-off (the reactive boundary)

• **Description:** An agreed path for what happens when a hunt **confirms a true positive**: it becomes an **incident** and enters your reactive incident-response process — the companion *M365 & Azure Incident Response & Escalation Runbook*. Hunters do **not** take response actions themselves.

• **Why required:** Hunting is read-only by design. A confirmed finding must reach responders through a clean, documented hand-off — not an ad-hoc scramble, and not a hunter improvising containment without authority or a change record.

• **How to verify:** Confirm the hand-off is written down (who to notify, how to create the incident, what evidence to attach) and that it references your IR runbook.

• **How to implement if missing:** Document the hand-off (see `03-implementation.md`, Phase 3): promote the bookmark to a Sentinel **incident**, attach the evidence, and trigger the IR runbook's intake.

R7. The PowerShell toolchain

• **Description:** A workstation with **Microsoft.Graph** (`Microsoft.Graph.Security`) for Defender XDR advanced hunting via `Invoke-MgSecurityRunHuntingQuery`, and the **Az** modules (`Az.Accounts`, `Az.OperationalInsights`, `Az.SecurityInsights`) for Sentinel — signed in with an MFA-capable admin.

• **Why required:** The delivered tools run the KQL hunt library across both platforms, discover the hunting surface, validate the program, and report metrics using these modules.

• **How to verify:** `Get-Module Microsoft.Graph.Security, Az.OperationalInsights, Az.SecurityInsights -ListAvailable returns versions; Connect-MgGraph -Scopes ThreatHunting.Read.All and Connect-AzAccount` both succeed.

• **How to implement if missing:** `Install-Module Microsoft.Graph, Az -Scope CurrentUser`; sign in.

Consent the `ThreatHunting.Read.All` Graph scope for advanced hunting. For Government cloud, connect Graph with the matching national-cloud endpoint and Az with `-Environment AzureUSGovernment` — *verify the correct endpoints and scopes for your cloud.*

R8. Awareness of advanced-hunting limits (schema, retention, quotas)

• **Description:** An understanding that Defender XDR advanced hunting has a **defined schema** (table and column names change as products evolve), **~30-day retention**, and **per-query and per-tenant rate/row limits**; the `runHuntingQuery` API has its own throttling.

• **Why required:** Hunts that assume a column that was renamed, a lookback beyond retention, or a result set beyond the row cap will fail or silently truncate — and a throttled API run looks like a "broken" script when it is really a quota.

• **How to verify:** In advanced hunting, hover a table to see its current columns; run a small `| count` first; note any truncation or throttling messages. *Verify current schema, retention, and limits in your tenant.*

• **How to implement if missing:** Keep hunts within the retention window, scope them (time, device, identity) before widening, and let the runner (`THREATHUNT-RunHuntLibrary.ps1`) handle paging/back-off — it treats limits as expected, not as errors.

Global Requirements — Sentinel Solutions

These requirements are common to **every** Sentinel Solutions solution in the standard program model. They are the baseline Azure Monitor / Microsoft Sentinel platform prerequisites that any telemetry, detection, or SIEM engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named* `<Purpose> - <Scope> - <Stage>` *and reused verbatim throughout the engagement (for example* `Audit Telemetry - Tenant - Pilot`*).*

G1. An Azure subscription

• **Description:** An active Azure subscription in the tenant in scope, with billing enabled.

• **Why:** A Log Analytics workspace and Microsoft Sentinel are Azure resources billed against a subscription; nothing can be provisioned without one.

• **Verify:** Sign in to `https://portal.azure.com`, open `Subscriptions`, and confirm an active subscription resolves and shows status `Active`.

• **Implement-if-missing:** Create or associate an Azure subscription (Enterprise Agreement, CSP, or pay-as-you-go) and confirm billing is in place.

G2. A Log Analytics workspace

• **Description:** A Log Analytics workspace to receive and store ingested telemetry, sized to the data volume and retention in scope.

• **Why:** Microsoft Sentinel runs on top of a Log Analytics workspace; all log data lands here and all queries/analytics run against it.

• **Why region matters:** the workspace region determines data residency and affects egress; pick one deliberately before ingestion starts (it cannot be moved later).

• **Verify:** `portal.azure.com` → `Log Analytics workspaces` → confirm the target workspace exists, note its region and resource group.

• **Implement-if-missing:** Create a workspace named `Security Telemetry - Tenant - Workspace`; choose the region for residency; select the pricing tier that fits expected daily ingestion.

G3. Microsoft Sentinel enabled on the workspace

• **Description:** Microsoft Sentinel added to (enabled on) the Log Analytics workspace in scope.

• **Why:** Connectors, analytics rules, hunting, and incidents are Sentinel features layered on the workspace; without Sentinel enabled you have logs but no SIEM.

• **Verify:** `portal.azure.com` → `Microsoft Sentinel` → confirm the workspace is listed.

• **Implement-if-missing:** `Microsoft Sentinel` → `Create` → select the workspace → `Add`.

G4. Administrative roles (least privilege)

• **Description:** Appropriately scoped roles for the work — Microsoft Sentinel Contributor (or Log Analytics Contributor) on the workspace for configuration, plus the tenant roles needed to set diagnostic settings on the sources (for example a role that can edit Entra ID diagnostic settings).

• **Why:** Each configuration action requires sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** On the workspace/resource group, `Access control (IAM)` → confirm the operating accounts hold the required roles. In Entra → `Roles & admins`, confirm the role that can edit diagnostic settings.

• **Implement-if-missing:** Assign the minimum roles needed at the resource-group or workspace scope; prefer eligible (PIM) activation over standing access where available.

G5. Ingestion and retention cost plan

• **Description:** A documented estimate of daily ingestion (GB/day) and a retention decision (interactive retention vs long-term/archive), with a budget owner.

• **Why:** Sentinel and Log Analytics are billed primarily on data ingested and data retained; identity and device audit logs can be high-volume, so cost must be planned before connectors are switched on.

• **Verify:** Confirm a written estimate exists (sources × expected GB/day) and that retention periods are agreed per table or workspace.

• **Implement-if-missing:** Estimate volume from the source log categories in scope; choose pay-as-you-go or a commitment tier; set workspace/table retention; record the budget and owner.

G6. Diagnostic settings capability on each source

• **Description:** The ability to create diagnostic settings (or enable the relevant data connector) on every source whose telemetry is in scope.

• **Why:** Telemetry only flows to the workspace when a diagnostic setting (or connector) routes the source's log categories to it; this is the switch that turns the pipeline on.

• **Verify:** For each source, confirm a diagnostic setting can be created and that no existing setting already routes the same categories elsewhere (to avoid duplicate ingestion and double billing).

• **Implement-if-missing:** Plan one named diagnostic setting per source that forwards the in-scope categories to the workspace (named per G's convention).

G7. Change control & rollback posture

• **Description:** An approved change process with a scheduled window, plus a documented rollback (which diagnostic settings/connectors/analytics rules were added) before any change.

• **Why:** Enabling connectors and analytics rules changes ingestion volume (and therefore cost) and can generate incidents; controlled changes and a recorded back-out keep the pipeline safe and auditable.

• **Verify:** Confirm an approved change record and a written list of objects to be created (so they can be removed cleanly).

• **Implement-if-missing:** Stand up a lightweight change/approval process; capture the pre-change configuration; document the rollback (delete the named diagnostic setting / disable the connector / disable the rule).

Implementation — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

This is the ordered, do-this-then-that guide. It follows the standard model (**Assess** → **Configure** → **Pilot** → **Validate** → **Expand**) and is **read-only and pilot-first**: you assess the hunting surface, stand up the library and RBAC, run a pilot batch of hypothesis-driven hunts, document findings, validate that a successful hunt promotes into a detection, then expand across the ATT&CK matrix on a cadence. **Every hunt is read-only** — you never isolate, disable, or remediate from here. Every object is named verbatim so the scripts, testing, and troubleshooting line up. Replace `contoso.com` / `contoso.onmicrosoft.com` and the sample names with your own.

> ***Portal note.*** *Advanced hunting is reached from the* ***Defender portal*** *(`security.microsoft.com`) → **Hunting** → **Advanced hunting**; custom detection rules under **Hunting** → **Custom detection rules**. Sentinel is reached from `portal.azure.com` → **Microsoft Sentinel** (or the unified Defender portal once onboarded); its hunting experience is under **Threat management** → **Hunting**, saved queries under **Hunting** → **Queries**, evidence under **Bookmarks**, and detections under **Analytics**. Portal paths, the advanced-hunting **schema**, retention, and query limits evolve — **verify against your tenant.***
>
> ***Read-only boundary (do not cross it here).*** *This solution *finds*. When a hunt confirms a true positive, **document it and hand it off** to the reactive process (the companion *M365 & Azure Incident Response & Escalation Runbook*) — see **Phase 3**. Do **not** take response actions (device isolation, account disablement, mailbox purge) from a hunt; those require the IR runbook's authority and change control.*

Object naming (used verbatim throughout)

Object	Name	MITRE (example)
Read-only hunter RBAC group	`Threat Hunters - Tenant - Group`	—

Object	Name	MITRE (example)
Reusable KQL hunt library	`Hunt Library - Tenant - Main (THREATHUNT-HuntLibrary.kql)`	—
Hunt register (findings home)	`Hunt Register - Tenant - Main`	—
Recurring hunt cadence	`Hunt Cadence - Tenant - Monthly`	—
Hunt — LOLBin / script-interpreter abuse	`Hunt-LOLBins-Devices-Pilot`	T1059 (Execution)
Hunt — anomalous parent→child process	`Hunt-RareParentChild-Devices-Pilot`	T1055 / T1036 (Defense Evasion)
Hunt — suspicious inbox / forwarding rules	`Hunt-InboxRules-M365-Pilot`	T1114 / T1564 (Collection)
Hunt — rare service-account sign-in	`Hunt-RareServiceSignin-Entra-Pilot`	T1078 (Valid Accounts)
Hunt — beaconing / rare egress	`Hunt-BeaconEgress-Network-Pilot`	T1071 (Command & Control)
Finding bookmark	`Bookmark-<Hunt>-Finding`	inherits hunt's technique
Promoted scheduled analytics rule (Sentinel)	`Rule-<Hunt>-Prod`	inherits hunt's technique
Promoted custom detection (Defender XDR)	`CD-<Hunt>-Prod`	inherits hunt's technique

(The five named hunts are worked examples that ship in the KQL library — one per ATT&CK tactic band. Apply the same hypothesis→scope→run→document→promote pattern to the rest of your prioritized backlog.)

Phase 0 — Assess (is the hunting surface usable?)

1. **Confirm the baseline (G1–G7) and R1–R2** — Defender XDR onboarded and the advanced-hunting tables

have data; the Sentinel workspace is connected and its hunting tables have data.

1. **Inventory the hunting surface.** Run `THREATHUNT-Discovery.ps1` (read-only): it reports which

advanced-hunting tables and which Sentinel tables have recent data, and lists existing Sentinel **hunt queries**, **bookmarks**, and **custom detection rules** — so you start from reality, not a blank page.

1. **Load the priority (R4).** Take leadership's ranked ATT&CK techniques and pick the **pilot batch** —

the five shipped hunts map to five common priorities (Execution, Defense Evasion, Collection, Valid Accounts, C2); reorder to match your backlog.

• **Gate 0:** advanced-hunting and Sentinel tables have data, the surface is inventoried, and a

prioritized pilot batch is chosen → continue.

Phase 1 — Configure (RBAC, library, register, cadence)

Step 1 — Stand up read-only hunter access (R3)

1. Entra → **Groups** → **New group** → name it exactly `Threat Hunters - Tenant - Group`; add the

hunters.

1. On the workspace / `RG-Security-Sentinel` → **Access control (IAM)** → **Add role assignment** → assign

Microsoft Sentinel Reader to `Threat Hunters - Tenant - Group`.

1. In the Defender portal → **Permissions** (or Entra role) → grant the group **Security Reader** so

they can run advanced hunting without any change rights.

1. Keep a **separate, smaller** admin set (Sentinel Contributor / Defender security-settings) for

promotion only (Phase 4). Hunters hunt; a few admins promote.

Step 2 — Import the reusable KQL hunt library

1. Open `THREATHUNT-HuntLibrary.kql` (`Hunt Library - Tenant - Main`) — it contains the five named

hunts, each with a header block (name, hypothesis, MITRE tactic/technique, target surface XDR-vs-Sentinel, and a promotion note).

1. **Save each hunt as a reusable query** so the team hunts from a shelf, not from memory:

• **Sentinel-side hunts** → Sentinel → **Hunting** → **Queries** → **New query** → paste the KQL, set the

Tactics & techniques tags to match the header, and name it exactly (e.g. `Hunt-RareServiceSignin-Entra-Pilot`).

• **Defender-XDR-side hunts** → advanced hunting → paste the KQL → **Save as** → name it exactly

(e.g. `Hunt-LOLBins-Devices-Pilot`).

1. Confirm the tags are set — the technique tag is what makes the library a **coverage map** later.

Step 3 — Create the hunt register and set the cadence

1. Create `Hunt Register - Tenant - Main` (a doc/wiki/list). For every hunt, capture: **hypothesis, scope (time/assets/identities), the exact KQL, outcome (found / not found / follow-up), and the decision (promote / hand-off / close)**.

1. Establish the bookmark convention `Bookmark-<Hunt>-Finding` so evidence rows are named

consistently.

1. Schedule `Hunt Cadence - Tenant - Monthly` — a recurring hunt cycle (monthly is a sensible default; high-risk techniques may warrant more often). Put it on the security calendar with an owner.

• **Gate 1:** RBAC group in place (read-only), library saved and tagged on both surfaces, register created, cadence scheduled → continue.

Phase 2 — Pilot (run hypothesis-driven hunts, read-only, and document)

Run the pilot batch. For **each** hunt, follow the same loop; the whole batch can be executed at once with `THREATHUNT-RunHuntLibrary.ps1` (read-only — it runs every `.kql` hunt across the right surface and reports row counts), then triage the hits interactively.

1. **State the hypothesis** (from the hunt header) in the register — e.g. *"An attacker is using a

built-in LOLBin (e.g. `mshta.exe`, `rundll32.exe`) to execute code; unusual command lines will surface it."* (`Hunt-LOLBins-Devices-Pilot`, **T1059**).

1. **Scope it.** Start narrow — a time window within retention, a subset of high-value devices or

identities — then widen. Advanced hunting is ~30-day retention; Sentinel may hold longer (**R8**).

1. **Run the KQL.**

• Interactively: paste the saved query in advanced hunting (Defender-side hunts) or Sentinel **Logs**

(Sentinel-side hunts).

• In bulk: `pwsh -File .\THREATHUNT-RunHuntLibrary.ps1 -ResourceGroup RG-Security-Sentinel

-WorkspaceName "Security Telemetry - Tenant - Workspace"` — it executes the library on both platforms and prints per-hunt result counts. **It never takes action; it only reads.**

1. **Triage the results** — benign / suspicious / confirmed. Most hunt hits are benign-but-interesting;

that is normal and still valuable (it tunes the next run).

1. **Document the outcome and preserve evidence.**

• Record the outcome + decision in `Hunt Register - Tenant - Main`.

• For anything suspicious/confirmed, in Sentinel **Hunting/Logs** → **select the row(s)** → **Add bookmark**,

naming it `Bookmark-<Hunt>-Finding` (e.g. `Bookmark-Hunt-InboxRules-M365-Pilot-Finding`), and map its **entities** (account, host, IP). For a Defender-side finding, capture the evidence and link it in the register.

1. Repeat for the other pilot hunts: `Hunt-RareParentChild-Devices-Pilot` (**T1055/T1036**),

`Hunt-InboxRules-M365-Pilot` (**T1114**), `Hunt-RareServiceSignin-Entra-Pilot` (**T1078**), `Hunt-BeaconEgress-Network-Pilot` (**T1071**).

• **Gate 2:** every pilot hunt has run, been triaged, and been documented (outcome + decision), with

bookmarks for any finding → continue.

Phase 3 — Hand off confirmed findings to incident response (the reactive boundary)

For any hunt that **confirms a true positive**, hand it off — do **not** respond from here.

1. In Sentinel, from the finding's bookmark, **create an incident** (Bookmarks → select →

Create incident, or add the bookmark to an existing incident) so the finding enters the reactive queue with its evidence and entities attached.

1. Attach the register entry (hypothesis, query, scope) and the bookmark(s) so responders inherit the

full context.

1. **Trigger the *M365 & Azure Incident Response & Escalation Runbook*** intake — that solution owns

triage, containment (device isolation, account disablement, mailbox purge), eradication, and escalation. This solution's job ends at a clean, evidenced hand-off.

• **Gate 3:** confirmed findings are incidents in the reactive queue with evidence attached; no

response action was taken from within the hunt → continue.

Phase 4 — Validate (coherent results + a clean promotion)

1. Run `THREATHUNT-Validate.ps1` — pass/fail lines aligned to `05-testing.md`: the hunter RBAC group

is read-only, the library is saved/tagged on both surfaces, the expected tables have data, the pilot hunts produced documented outcomes, and **at least one successful hunt has been promoted** to a scheduled analytics rule / custom detection.

1. **Promote a successful hunt into a permanent detection** (elevated admin, not a hunter):

• **Defender XDR side:** advanced hunting → open the saved hunt (e.g. `Hunt-LOLBins-Devices-Pilot`) →

Create custom detection rule → name it `CD-LOLBins-Devices-Prod`, set frequency, impacted entities, and the **ATT&CK technique** tag. *(Custom-detection availability/limits can differ by license and cloud — verify.)*

• **Sentinel side:** Sentinel → **Hunting** → **Queries** → **the saved hunt** → **Create analytics rule** (or

Analytics → **Create** → **Scheduled rule** with the hunt's KQL) → name it `Rule-RareServiceSignin-Entra-Prod`, set schedule, entity mappings, and MITRE tags. **Start it in a low-impact posture** (incident creation disabled) and **tune it** using the *Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage* solution before it creates incidents the SOC trusts.

• **Gate 4 (go/no-go):** validation lines PASS and at least one hunt is promoted (as a `-Prod`

detection) → the program is working; expand.

Phase 5 — Expand (grow the library, run the cadence)

1. **Promote the hunts worth keeping.** Any hunt that repeatedly finds real, actionable activity becomes

a `-Prod` detection (name per the table). Any hunt that only ever returns benign noise is either refined or retired — record which in the register.

1. **Grow the library across the ATT&CK matrix.** Add hunts for the next priority techniques (the

library file has a template header to copy). Tag every new hunt so the coverage map keeps growing.

1. **Run** `Hunt Cadence - Tenant - Monthly`**.** Each cycle: pick the next backlog items, run

`THREATHUNT-RunHuntLibrary.ps1`, triage, document, promote/hand-off, and re-report with `THREATHUNT-Report.ps1`. Hunting is a **program**, not a one-time task.

1. **Track the trend.** A growing set of tagged hunts, findings that turn into detections, and shrinking time-to-find are the signs the program is working — `THREATHUNT-Report.ps1` surfaces these.

Rollback (for any step)

- **A saved hunt query:** delete it (Hunting → Queries / advanced hunting → the saved query → Delete) — it stops appearing on the shelf; nothing else changes (queries are read-only).
- **A promoted detection:** disable it (Sentinel Analytics → the rule → **Disable**, or Defender → Custom detection rules → the rule → **Turn off**) — it stops evaluating immediately; the saved hunt remains.
- **The hunter RBAC group:** remove the role assignment to revoke hunting access cleanly (the group and its history remain for audit).
- **A bookmark / incident hand-off:** a bookmark can be deleted; an incident created from it should be **closed with a classification** (do not silently delete) so the reactive record stays intact.

Testing & Validation — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

Work these gates **in order**. Each is **pass/fail**: do not advance a phase until its gate passes. Run `THREATHUNT-Validate.ps1` for a consolidated pass/fail summary, `THREATHUNT-Discovery.ps1` for the hunting-surface inventory, `THREATHUNT-RunHuntLibrary.ps1` to execute the library across both platforms, and `THREATHUNT-Report.ps1` for hunt-program metrics. KQL runs in advanced hunting (`security.microsoft.com` → **Hunting** → **Advanced hunting**) and in Sentinel → **Logs**. **Every check is read-only** — testing this solution never changes or contains anything.

Gate 1 — The hunting surface has data (both platforms)

- **Check (KQL):** advanced hunting `DeviceProcessEvents | where Timestamp > ago(24h) | count` → non-zero; Sentinel `SigninLogs | where TimeGenerated > ago(24h) | count` → non-zero (repeat per table your pilot hunts need).
- **Automated:** `THREATHUNT-Discovery.ps1` reports which advanced-hunting and Sentinel tables have recent data.
- **Pass:** every table a pilot hunt depends on has recent data on the platform that hunt targets.

Fail: onboard/connect the source first (R1/R2) — do not hunt an empty table.

Gate 2 — Read-only hunter access is in place (and only read-only)

• **Check:** Entra → **Groups** shows `Threat Hunters - Tenant - Group`; the workspace **IAM** shows it

holding **Microsoft Sentinel Reader**; the Defender portal shows members with **Security Reader**. Promotion rights (Sentinel Contributor / Defender security settings) belong to a **separate, smaller** admin set — not to the hunter group.

• **Automated:** `THREATHUNT-Validate.ps1` reports the group's workspace role and flags it if the hunter

group holds any write/contributor role.

• **Pass:** hunters can query both surfaces and can change nothing. **Fail:** correct the role

assignments (Phase 1, Step 1).

Gate 3 — The KQL hunt library is saved and MITRE-tagged on both surfaces

• **Check:** Sentinel → **Hunting** → **Queries** lists the Sentinel-side hunts (e.g.

`Hunt-RareServiceSignin-Entra-Pilot`) **with Tactics & techniques tags**; advanced hunting → saved queries lists the Defender-side hunts (e.g. `Hunt-LOLBins-Devices-Pilot`).

• **Automated:** `THREATHUNT-Discovery.ps1` lists existing Sentinel hunt queries; `THREATHUNT-Validate.ps1`

checks the expected hunt names are present and tagged.

• **Pass:** all five pilot hunts are saved on the correct surface and tagged to a technique. **Fail:**

import/tag them (Phase 1, Step 2).

Gate 4 — The library runs clean on both platforms

• **Check:** `THREATHUNT-RunHuntLibrary.ps1` executes every hunt in `THREATHUNT-HuntLibrary.kql` —

Defender-side hunts via `Invoke-MgSecurityRunHuntingQuery`, Sentinel-side via `Invoke-AzOperationalInsightsQuery` — and prints a per-hunt **row count** with no query errors (schema/columns resolve).

• **Pass:** every hunt returns a result set (zero rows is a valid result — it means *not found this

window*), no hunt errors on a missing column or a lookback beyond retention. **Fail:** fix the failing hunt's schema/lookback (R8) — see troubleshooting.

Gate 5 — Pilot hunts are documented (hypothesis → outcome → decision)

• **Check:** `Hunt Register - Tenant - Main` has an entry per pilot hunt with **hypothesis, scope, the

exact KQL, outcome (found / not found / follow-up), and decision (promote / hand-off / close)**.

• **Pass:** every pilot hunt has a complete register entry. **Fail:** a hunt with no documented outcome

is not finished — complete the entry (Phase 2, Step 5).

Gate 6 — Findings are preserved as bookmarks with entities

• **Check:** Sentinel → **Bookmarks** — any suspicious/confirmed finding is bookmarked as `Bookmark-<Hunt>-Finding` with **entities mapped** (account/host/IP).

• **Pass:** every finding worth keeping is a named, entity-mapped bookmark (evidence survives the query window). **Fail:** bookmark and map the finding (Phase 2, Step 5).

Gate 7 — Confirmed true positives are handed off (not acted on here)

• **Check:** each confirmed finding is a Sentinel **incident** (created from its bookmark) with evidence attached, and the *M365 & Azure Incident Response & Escalation Runbook* intake was triggered. No response action (isolation, disablement, purge) was taken from within the hunt.

• **Pass:** confirmed findings are incidents in the reactive queue with full context; the read-only boundary held. **Fail:** create the incident and hand off (Phase 3) — do not respond from the hunt.

Gate 8 — At least one successful hunt is promoted to a detection

• **Check:** a successful hunt exists as a permanent detection — a Defender **custom detection rule** `CD-<Hunt>-Prod`, or a Sentinel **scheduled analytics rule** `Rule-<Hunt>-Prod` (started in low-impact posture and pointed at the tuning solution).

• **Automated:** `THREATHUNT-Validate.ps1` checks for at least one promoted `-Prod` detection whose logic traces back to a library hunt.

• **Pass:** the program has captured at least one win as standing coverage. **Fail:** promote a reliable hunt (Phase 4, Step 2).

Gate 9 — The cadence and trend are established

• **Check:** `Hunt Cadence - Tenant - Monthly` is scheduled with an owner; `THREATHUNT-Report.ps1` shows hunts-run, findings, hand-offs, and promotions for the cycle.

• **Pass:** the program runs on a cadence and its metrics are visible (a growing tagged-hunt count and promotions are the signs it is working). **Fail:** schedule the cadence and generate the first report.

Consolidated pass criteria

The hunt program is **stood up and working** when **Gates 1–9 pass**: the surface has data, hunter access is read-only, the library is saved and MITRE-tagged, it runs clean on both platforms, pilot hunts are documented, findings are bookmarked, confirmed true positives are **handed off** (never acted on from the hunt), at least one hunt is **promoted** to a detection, and the program runs on a cadence with visible metrics. Then repeat the loop for the next batch of techniques.

Troubleshooting — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

Common failures, each as **symptom** → **cause** → **fix**. Start with `THREATHUNT-Discovery.ps1` (surface inventory + table freshness), `THREATHUNT-RunHuntLibrary.ps1` (runs the whole library and shows which hunt fails), and `THREATHUNT-Validate.ps1` (the program gates) — most issues here surface in one of the three. Every tool is **read-only**; none of these fixes involves a response action.

A hunt returns nothing (and you expected activity)

- **Symptom:** A hunt runs cleanly but returns zero rows, even for behavior you believe exists.
- **Cause:** The target table has no data on that platform (source not onboarded/connected), the

lookback is beyond retention (advanced hunting is ~30 days), the filter is too tight, or you ran a Defender-side hunt against Sentinel (or vice-versa) where that table does not exist.

- **Fix:** Confirm the table has data on the **right** surface (`<Table> | where Timestamp/TimeGenerated

ago(1d) | count`; Gate 1 / R1–R2). Widen the window (within retention) and loosen the filter to

confirm the logic returns *something*, then re-tighten. Remember: a true null over a valid window is a legitimate "not found this window" — record it in the register.

A hunt returns far too much to triage

- **Symptom:** A hunt returns thousands of rows, most of them obviously benign.
- **Cause:** The hypothesis is too broad, no known-good baseline is excluded, or the window is too wide

for a first pass.

- **Fix:** Scope narrower (shorter window, high-value assets/identities first). Add a baseline exclusion

(e.g. summarize by a rare command line / rare process count and keep only the uncommon). Iterate the KQL toward *rare and interesting*, not *everything*. Broad-and-benign is a signal to refine the hunt, not to promote it.

A hunt errors on a missing column or unknown table

- **Symptom:** The KQL fails with "unknown column" / "unresolved table" in advanced hunting or Sentinel.
- **Cause:** The advanced-hunting **schema evolves** — a table or column was renamed, or the column

exists on Defender XDR but not in the Sentinel copy of the table (or vice-versa). Some tables are preview/`*Beta` and differ by tenant.

- **Fix:** Hover the table in advanced hunting to see its **current** columns; adjust the hunt to the

live schema. Run the hunt on the surface whose schema it was written for (the library header notes XDR-vs-Sentinel). *Verify current schema in your tenant* — treat schema drift as expected maintenance of the library.

Results look truncated / a limit was hit

- **Symptom:** Advanced hunting warns about a result or time limit; the row count looks capped.
- **Cause:** Advanced hunting enforces **per-query row and time limits**; a hunt that returns more than the cap is truncated.
- **Fix:** Aggregate instead of returning raw rows (`summarize`/`count` first, then drill into the interesting buckets); narrow the window; add filters before the expensive operators. Keep hunts within retention (R8). *Verify current limits in your tenant.*

`Invoke-MgSecurityRunHuntingQuery` throttles or errors

- **Symptom:** `THREATHUNT-RunHuntLibrary.ps1` fails on the Defender-side hunts with a 429 / throttling or a permissions error.
- **Cause:** The `runHuntingQuery` API has its own rate limits, or the app/user lacks the `ThreatHunting.Read.All` scope, or (Gov cloud) the wrong Graph national-cloud endpoint was used.
- **Fix:** Let the runner back off and retry (it treats 429 as expected). Consent `Connect-MgGraph -Scopes ThreatHunting.Read.All`. For Government/national clouds, connect Graph to the matching endpoint (`Connect-MgGraph -Environment <Gov endpoint>`) — *verify the correct endpoint and scopes for your cloud* (R7).

The hunter group can change things (or cannot query)

- **Symptom:** Hunters can disable rules / isolate devices (too much), **or** hunters get "access denied" running advanced hunting (too little).
- **Cause:** `Threat Hunters - Tenant - Group` was granted a **contributor/write** role (too broad), or it lacks **Security Reader** (Defender) / **Microsoft Sentinel Reader** (workspace) (too narrow).
- **Fix:** Reassign to **read-only** exactly: **Microsoft Sentinel Reader** on the workspace and **Security Reader** in Defender. Keep promotion rights in a **separate, smaller** admin set (Phase 1, Step 1 / R3). `THREATHUNT-Validate.ps1` flags a hunter group that holds a write role.

A bookmark loses its evidence or entities

- **Symptom:** A `Bookmark-<Hunt>-Finding` exists but has no usable entities, or the underlying row is gone.
- **Cause:** Entities were not mapped when the bookmark was created, or the raw data rolled off retention before it was bookmarked (advanced hunting ~30 days).
- **Fix:** Bookmark **promptly** and **map entities** (account/host/IP) at creation. For long-lived

evidence, prefer the Sentinel copy of the table (often longer retention) or export the row. A finding worth keeping should be bookmarked the moment it is triaged (Gate 6).

A confirmed true positive — but no clean hand-off

• **Symptom:** A hunt confirmed malicious activity but responders had to reconstruct the context, or a hunter started containing it themselves.

• **Cause:** The read-only boundary was crossed, or the finding was not turned into an incident with evidence attached.

• **Fix: Do not respond from the hunt.** Create a Sentinel **incident** from the bookmark, attach the register entry and evidence, and trigger the *M365 & Azure Incident Response & Escalation Runbook* intake (Phase 3 / Gate 7). Containment, disablement, and escalation happen **there**, with authority and change control.

A promoted detection floods the queue

• **Symptom:** A hunt was quiet when run by hand, but the promoted `Rule-<Hunt>-Prod` / `CD-<Hunt>-Prod` floods incidents once live.

• **Cause:** It was promoted straight to full incident creation without tuning — a manual hunt tolerates noise a running detection does not.

• **Fix:** Return the Sentinel rule to **low-impact posture** (incident creation disabled) and tune it in the *Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage* solution (allow-list → suppression → threshold → grouping) before it creates incidents. For a Defender custom detection, tune frequency/logic before relying on it.

PowerShell / connection errors

• **Symptom:** A script errors on connect, on the workspace, or on the Graph call.

• **Cause:** Wrong subscription/tenant context, missing `Microsoft.Graph.Security` or `Az.SecurityInsights`/`Az.OperationalInsights`, insufficient read role, or (Gov cloud) the wrong environment/endpoint.

• **Fix:** `Connect-AzAccount` + `Set-AzContext` (Sentinel) and `Connect-MgGraph -Scopes ThreatHunting.Read.All (Defender); Install-Module Microsoft.Graph, Az; confirm **Microsoft Sentinel Reader / Security Reader**. For Government cloud pass -Environment AzureUSGovernment` (Az) and the matching Graph national-cloud endpoint (R7) — *verify endpoints for your cloud.*

Best Practices — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

Keep-it-clean guidance for a hunt program that stays proactive, disciplined, and current over time.

Hunt from a hypothesis, not a hunch

• **Every hunt starts with a written hypothesis.** "If technique *X* were happening, this query would surface it." A query with no hypothesis is a fishing trip you cannot document, repeat, or learn from.

• **Scope narrow, then widen.** Start within retention and on high-value assets/identities, confirm the logic returns *something* sane, then broaden. It is faster, cheaper, and avoids hitting query limits (R8).

• **A null result is still a result.** "Not found this window" is valuable and belongs in the register — it records where you have looked, so effort goes to unhunted ground next cycle.

Hunt against a map (MITRE ATT&CK)

• **Tag every hunt to a technique.** The tags turn the library into a **coverage map**: you can see which tactics you actively hunt and which you have never looked at — and steer the backlog to the gaps.

• **Prioritize by risk, not by ease.** Which techniques matter most is a **leadership decision (R4)**; hunt the crown-jewel-relevant techniques first, not the ones with the tidiest query.

• **Cover breadth *and* depth.** One shallow hunt per tactic is thin coverage. For high-risk techniques, run several complementary hunts.

Stay read-only — hunting finds, it does not act

• **Never take a response action from a hunt.** Isolation, account disablement, mailbox purge, and escalation belong to the **reactive** process (*M365 & Azure Incident Response & Escalation Runbook*). A hunter improvising containment has neither the authority nor the change record for it.

• **Separate hunt (read) from promote (write).** Give hunters read-only access; keep detection creation with a smaller admin set. Least privilege keeps the program safe and auditable.

• **Hand off cleanly.** A confirmed true positive becomes an **incident with evidence attached**, not a chat message — so responders inherit the full context and nothing is lost in translation.

Document so the knowledge compounds

• **The register is the program.** Hypothesis, scope, query, outcome, decision — per hunt. A hunt whose result lives in a browser tab is a hunt you will pay to run again.

• **Bookmark the evidence.** Advanced hunting is ~30-day retention; a `Bookmark-<Hunt>-Finding` (with entities mapped) preserves the row after the window rolls off.

• **Version the library.** Keep `Hunt Library - Tenant - Main` (`THREATHUNT-HuntLibrary.kql`) in source

control alongside the promoted detections, so a good query is reviewable, diffable, and portable.

Promote wins into permanent detections

- **A hunt that keeps finding real activity should stop being a hunt.** Promote it to a **custom detection **(Defender XDR) or a** scheduled analytics rule** (Sentinel) so it runs automatically forever — that is how a program compounds.
- **Promote into a low-impact posture, then tune.** A promoted Sentinel rule starts with incident creation disabled and is tuned for false positives in the *Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage* solution before the SOC trusts it. A noisy auto-detection is worse than a manual hunt.
- **Retire what only ever finds noise.** Not every hunt deserves promotion; record the decision so the library stays sharp.

Run it as a program, on a cadence

- **Loop, don't launch-and-forget.** Assess → configure → pilot → validate → expand, then take the next batch of techniques on `Hunt Cadence - Tenant - Monthly`. Adversary behavior and your environment both drift; the cadence keeps hunting honest.
- **Feed threat intel into the backlog.** New intel (a campaign, a new TTP) should spawn a hypothesis and a hunt — the fastest way to know whether *that* threat is already in your environment.
- **Watch the trend.** Growing tagged-hunt coverage, findings that become detections, and shrinking time-to-find are the signs the program is maturing — `THREATHUNT-Report.ps1` surfaces them.

Where to go next

With a proactive hunt program running, the natural next steps in this catalog are:

- **Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage** — tune the detections you promote from hunts so they are trustworthy and low-noise (the promotion target of this solution).
- **M365 & Azure Incident Response & Escalation Runbook** — the reactive counterpart that works the incidents your hunts hand off (containment, eradication, escalation).
- **Sentinel SOAR / automation playbooks** — automate enrichment and response for the detections your hunts create.
- **Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion** — add the sources that would let you hunt techniques you currently cannot see.

Deliverables Summary — Threat Hunting with KQL: Microsoft Defender XDR & Sentinel

Everything included in this solution, and what each piece is for.

Documentation

Document	Purpose
Executive Summary (`executive-summary.md`)	One-page business framing: the reactive-only blind spot and the knowledge-that-evaporates problem, the proactive KQL hunt-program solution, and the boundary (finds and promotes; does **not** respond — that is the IR runbook).
Solution Overview (`01-solution-overview.md`)	What it is, how the technology works (advanced hunting vs Sentinel hunting, KQL, MITRE alignment, hypothesis-driven method, bookmarks/register, hunt→detection promotion, the read-only hand-off), the Assess→Configure→Pilot→Validate→Expand model, scope, out-of-scope, and connections used.
Requirements (`02-requirements.md`)	Solution-specific prerequisites (Defender XDR onboarded, connected Sentinel workspace, read-only hunter RBAC, an agreed hunt backlog, a findings home, a hunt-to-incident hand-off, the toolchain, and advanced-hunting limits) — layered on the shared `global-requirements-sentinel-solutions.md` (G1–G7).
Implementation (`03-implementation.md`)	Exact, ordered build: assess the surface, stand up read-only RBAC + the KQL library + register + cadence, run pilot hunts read-only and document them, hand off confirmed findings to IR, validate and promote a win, then expand on a cadence — every object named, with rollback.
Testing & Validation (`05-testing.md`)	Nine pass/fail gates from surface-has-data and read-only-access through library-runs-clean, findings-documented-and-bookmarked, confirmed-true-positives-handed-off, at-least-one-hunt-promoted, and cadence-established, referencing the scripts by name.
Troubleshooting (`06-troubleshooting.md`)	The common failure modes (hunt returns nothing/too much, schema/column errors, retention/limit truncation, Graph `runHuntingQuery` throttling, RBAC too broad or too narrow, bookmark/entity issues, promotion floods, Gov-cloud/endpoint errors) as symptom → cause → fix.

Document	Purpose
Best Practices (`07-best-practices.md`)	Hunt-from-a-hypothesis, hunt-against-a-map, stay-read-only, document-so-knowledge-compounds, promote-wins, run-it-as-a-program — plus where to go next in the catalog.

Hunt library & PowerShell tools (5)

All tools are **read-only** (they run KQL and report — no response actions, **no credentials in code**) and target both platforms: **Microsoft.Graph** (`Microsoft.Graph.Security` — `Invoke-MgSecurityRunHuntingQuery`) for Defender XDR advanced hunting, and the **Az** modules (`Az.Accounts`, `Az.OperationalInsights`, `Az.SecurityInsights`) for Sentinel. Government-cloud tenants pass the matching endpoint/environment — *verify endpoints and scopes for your cloud.*

File	Type	What it does
`THREATHUNT-HuntLibrary.kql`	KQL library	`Hunt Library - Tenant - Main` — the **MITRE ATT&CK-aligned reusable hunt library**: real, ready-to-run KQL hunts, each with a header block (name, hypothesis, tactic/technique, target surface XDR-vs-Sentinel, and a promotion note), plus a template header to add more. The core deliverable the whole program hunts from.
`THREATHUNT-Discovery.ps1`	Read-only	Inventories the hunting surface: which advanced-hunting tables and which Sentinel tables have recent data, and the existing Sentinel hunt queries, bookmarks, and custom detection rules — so hunting starts from reality. Changes nothing.
`THREATHUNT-RunHuntLibrary.ps1`	Read-only	Executes every hunt in `THREATHUNT-HuntLibrary.kql` across the correct platform (Defender via `runHuntingQuery`, Sentinel via the Log Analytics query API) and reports a per-hunt row count. The automation that runs the whole library in one pass. **Runs queries only — it never acts.**

File	Type	What it does
`THREATHUNT-Validate.ps1`	Read-only	Runs the `05-testing.md` gates as pass/fail lines: surface has data, the hunter RBAC group is **read-only** (flags any write role), the expected hunts are saved/tagged, pilot hunts documented, and at least one successful hunt **promoted** to a `-Prod` detection.
`THREATHUNT-Report.ps1`	Read-only	Hunt-program metrics: hunts run over the window, findings/bookmarks, incidents handed off, and promotions to detections, plus a coverage view of which ATT&CK techniques are actively hunted. Exports to CSV — the trend that shows the program maturing.

Named objects (verbatim — used across docs, scripts, and tests)

- **Read-only hunter RBAC group:** `Threat Hunters - Tenant - Group`
- **Reusable KQL hunt library:** `Hunt Library - Tenant - Main` (`THREATHUNT-HuntLibrary.kql`)
- **Findings home:** `Hunt Register - Tenant - Main`; bookmarks `Bookmark-<Hunt>-Finding`
- **Cadence:** `Hunt Cadence - Tenant - Monthly`
- **Pilot hunts (worked examples in the library):**
- `Hunt-LOLBins-Devices-Pilot` — **T1059** Command and Scripting Interpreter (Execution), Defender XDR
- `Hunt-RareParentChild-Devices-Pilot` — **T1055 / T1036** (Defense Evasion), Defender XDR
- `Hunt-InboxRules-M365-Pilot` — **T1114 / T1564** (Collection), Defender XDR / Sentinel
- `Hunt-RareServiceSignin-Entra-Pilot` — **T1078** Valid Accounts, Sentinel
- `Hunt-BeaconEgress-Network-Pilot` — **T1071** Application Layer Protocol (Command & Control), Defender XDR
- **Promoted detections:** `Rule-<Hunt>-Prod` (Sentinel scheduled rule), `CD-<Hunt>-Prod` (Defender custom detection)

What this solution deliberately does NOT do

- It does **not** respond — no isolation, account disablement, mailbox purge, eradication, or escalation.

Those are the companion *M365 & Azure Incident Response & Escalation Runbook*; this solution hands confirmed findings **to** it and stops.

• It does **not** connect data sources or size the workspace — that is the *Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion* solution; a hunt only finds what is ingested.

• It does **not** fully tune the detections it promotes — the *Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage* solution owns rule tuning; this solution performs the promotion.

• It does **not** decide which techniques matter most or your risk tolerance — those are security-leadership determinations that prioritize the hunt backlog.

• It does **not** invent schema, retention, licensing, or limit facts — where availability may differ by tenant/cloud/license it says **verify in your tenant**.

How it is delivered

This solution is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a formatted Word document and PDF, plus the KQL hunt library and all four PowerShell tools, in a single ZIP (`THREATHUNT-Complete-Solution.zip`) — yours to keep and reuse on every tenant and every hunt cycle.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

THREATHUNT-Discovery.ps1

Purpose: Establishes reality before hunting: which Defender XDR advanced-hunting tables and which Sentinel workspace tables have recent data, plus the existing Sentinel hunt queries, bookmarks, and custom detection rules. Read-only — it changes nothing and takes no response action. HOW TO RUN Defender XDR side (advanced hunting via Microsoft Graph): Connect-MgGraph -Scopes ThreatHunting.Read.All Sentinel side (Az): Connect-AzAccount

How to run: Defender XDR side (advanced hunting via Microsoft Graph): Connect-MgGraph -Scopes ThreatHunting.Read.All Sentinel side (Az): Connect-AzAccount

THREATHUNT-HuntLibrary.kql

THREATHUNT-Report.ps1

Purpose: Reports the health and trend of the hunt program so you can see it maturing: bookmarks (preserved findings) over the window, incidents that carry a hunt bookmark (hand-offs to the reactive process), promoted -Prod detections (hunts turned into standing coverage), and a coverage view of which ATT&CK tactics the saved hunt library actively covers. Read-only — it reads and reports; it changes nothing and takes no response action. HOW TO RUN Connect-AzAccount

How to run: Connect-AzAccount

THREATHUNT-RunHuntLibrary.ps1

Purpose: Executes every hunt in "Hunt Library - Tenant - Main" (THREATHUNT-HuntLibrary.kql) on its target surface — Defender XDR advanced hunting via Microsoft Graph runHuntingQuery, or Microsoft Sentinel via the Log Analytics query API — and prints a per-hunt result count. This is the automation that runs the library in one pass so the team can triage hits. It RUNS QUERIES ONLY: it reads, counts, and reports; it NEVER takes a response action (no isolate/disable/purge). A zero-row hunt is a valid "not found this window" result, not a failure. HOW TO RUN Connect-MgGraph -Scopes ThreatHunting.Read.All # Defender XDR advanced hunting Connect-AzAccount # Sentinel

How to run: Connect-MgGraph -Scopes ThreatHunting.Read.All # Defender XDR advanced hunting Connect-AzAccount # Sentinel

THREATHUNT-Validate.ps1

Purpose: Consolidates the testing gates into a pass/fail summary: the hunting surface has data (both platforms), the hunter RBAC group is READ-ONLY (flags any write/contributor role), the expected hunts are saved and MITRE-tagged, and at least one successful hunt has been PROMOTED to a -Prod detection. Read-only — it verifies posture; it changes nothing and takes no response action. HOW TO RUN Connect-MgGraph -Scopes ThreatHunting.Read.All Connect-AzAccount

How to run: Connect-MgGraph -Scopes ThreatHunting.Read.All Connect-AzAccount